Comparison: ai - faraz

Average Similarity: ***40.42%*** Similarity ai: *45.67%* Similarity faraz: *36.25%*

File of ai	Starting Line - End Line	Length in ai	File of faraz	Starting Line - End Line	Length in faraz
AVL.cpp	{ "line": 4, "column": 0, "tokenListIndex": 277 } - { "line": 42, "column": 32, "tokenListIndex": 331 }	55	AVL.cpp	{ "line": 10, "column": 0, "tokenListIndex": 8 } - { "line": 50, "column": 46, "tokenListIndex": 62 }	55
BST.cpp	{ "line": 60, "column": 0, "tokenListIndex": 81 } - { "line": 75, "column": 4, "tokenListIndex": 111 }	31	BST.cpp	{ "line": 63, "column": 0, "tokenListIndex": 336 } - { "line": 81, "column": 4, "tokenListIndex": 366 }	31
FileExplorer.cpp	{ "line": 5, "column": 0, "tokenListIndex": 122 } - { "line": 28, "column": 4, "tokenListIndex": 147 }	26	FileExplorer.cpp	{ "line": 4, "column": 0, "tokenListIndex": 122 } - { "line": 26, "column": 63, "tokenListIndex": 147 }	26
BST.cpp	{ "line": 19, "column": 42, "tokenListIndex": 19 } - { "line": 36, "column": 4, "tokenListIndex": 43 }	25	BST.cpp	{ "line": 5, "column": 35, "tokenListIndex": 256 } - { "line": 23, "column": 4, "tokenListIndex": 280 }	25
AVL.cpp	{ "line": 46, "column": 67, "tokenListIndex": 338 } - { "line": 64, "column": 4, "tokenListIndex": 362 }	25	AVL.cpp	{ "line": 50, "column": 59, "tokenListIndex": 63 } - { "line": 67, "column": 4, "tokenListIndex": 87 }	25
FileExplorer.cpp	{ "line": 43, "column": 0, "tokenListIndex": 165 } - { "line": 49, "column": 4, "tokenListIndex": 176 }	12	FileExplorer.cpp	{ "line": 49, "column": 0, "tokenListIndex": 185 } - { "line": 55, "column": 4, "tokenListIndex": 196 }	12

Files of ai:

381 total tokens

AVL.cpp

75%

```
#include "AVL.hpp"
#include <algorithm>

int AVLIndex::height(TreeNode* N) {
    if (N == nullptr)
        return 0;
    return N->height;
}

int AVLIndex::getBalance(TreeNode* N) {
    if (N == nullptr)
        return 0;
    return height(N->left) - height(N->right);
}

TreeNode* AVLIndex::rightRotate(TreeNode* y) {
    TreeNode* x = y->left;
    TreeNode* T2 = x->right;

    x->right = y;
    y->left = T2;

    y->height = std::max(height(y->left), height(y->right)) + 1;
```

Files of faraz:

480 total tokens

AVL.cpp

66%

```
#include "AVL.hpp"

int getHigherVal(int v1, int v2) {
    if (v1 > v2) {
        return v1;
    }
    return v2;
}

int AVLIndex::getHeight(TreeNode* n) {
    if (n == nullptr) {
        return 0;
    }
    return n->height;
}

int AVLIndex::getBalance(TreeNode* n) {
    if (n == nullptr) {
        return 0;
    }
    return getHeight(n->left) - getHeight(n->right);
}
```

```
    y->height = std::max(height(y->left), height(y->right)) + 1;
    x->height = std::max(height(x->left), height(x->right)) + 1;

    return x;
}

TreeNode* AVLIndex::leftRotate(TreeNode* x) {
    TreeNode* y = x->right;
    TreeNode* T2 = y->left;

    y->left = x;
    x->right = T2;

    x->height = std::max(height(x->left), height(x->right)) + 1;
    y->height = std::max(height(y->left), height(y->right)) + 1;

    return y;
}

void AVLIndex::Add(const int i, const std::string &Key) {
    root = insert(root, Key, i);
}

TreeNode* AVLIndex::insert(TreeNode* node, const std::string& key, int index) {
    if (node == nullptr)
        return new TreeNode(key, index);

    if (key < node->key)
        node->left = insert(node->left, key, index);
    else if (key > node->key)
        node->right = insert(node->right, key, index);
    else {
        node->fileIndices.push_back(index);
        return node;
    }

    node->height = 1 + std::max(height(node->left), height(node->right));

    int balance = getBalance(node);

    // Left Left Case
    if (balance > 1 && key < node->left->key)
        return rightRotate(node);

    // Right Right Case
    if (balance < -1 && key > node->right->key)
        return leftRotate(node);

    // Left Right Case
    if (balance > 1 && key > node->left->key) {
        node->left = leftRotate(node->left);
        return rightRotate(node);
    }

    // Right Left Case
```

```
TreeNode* AVLIndex::rotateRight(TreeNode* parent) {
    TreeNode* child = parent->left;
    TreeNode* grandChild = child->right;

    child->right = parent;
    parent->left = grandChild;

    parent->height = getHigherVal(getHeight(parent->left), getHeight(parent->right)) + 1;
    child->height = getHigherVal(getHeight(child->left), getHeight(child->right)) + 1;

    return child;
}

TreeNode* AVLIndex::rotateLeft(TreeNode* parent) {
    TreeNode* child = parent->right;
    TreeNode* grandChild = child->left;

    child->left = parent;
    parent->right = grandChild;

    parent->height = getHigherVal(getHeight(parent->left), getHeight(parent->right)) + 1;
    child->height = getHigherVal(getHeight(child->left), getHeight(child->right)) + 1;

    return child;
}

TreeNode* AVLIndex::insert(TreeNode* current, int dataIdx, const string& searchKey) {
    if (current == nullptr) {
        return new TreeNode(searchKey, dataIdx);
    }

    if (searchKey < current->key) {
        current->left = insert(current->left, dataIdx, searchKey);
    } else if (searchKey > current->key) {
        current->right = insert(current->right, dataIdx, searchKey);
    } else {
        current->indices.push_back(dataIdx);
        return current;
    }

    current->height = 1 + getHigherVal(getHeight(current->left), getHeight(current->right));
    int factor = getBalance(current);

    if (factor > 1) {
        if (searchKey < current->left->key) {
            return rotateRight(current);
        } else if (searchKey > current->left->key) {
            current->left = rotateLeft(current->left);
```

```
    if (balance < -1 && key < node->right->key) {
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }

    return node;
}

```

BST.cpp 46%

```
#include "BST.hpp"
#include <iostream>

BSTIndex::BSTIndex() : root(nullptr) {}

BSTIndex::~BSTIndex() {
    deleteTree(root);
}

void BSTIndex::deleteTree(TreeNode* node) {
    if (node) {
        deleteTree(node->left);
        deleteTree(node->right);
        delete node;
    }
}

void BSTIndex::CreateIndex(std::vector<FileSystemEntry> &Data, const std::string
&IndexType) {
    for (int i = 0; i < (int)Data.size(); ++i) {
        if (IndexType == "Filename") {
            Add(i, Data[i].Filename);
        } else if (IndexType == "modifiedOn") {
            Add(i, Data[i].modifiedOn);
        }
    }
}

void BSTIndex::Add(const int i, const std::string &Key) {
    root = insert(root, Key, i);
}

TreeNode* BSTIndex::insert(TreeNode* node, const std::string& key, int index) {
    if (!node) {
        return new TreeNode(key, index);
    }
    if (key == node->key) {
        node->fileIndices.push_back(index);
    } else if (key < node->key) {
        node->left = insert(node->left, key, index);
    } else {
        node->right = insert(node->right, key, index);
    }
    return node;
```

```
            return rotateRight(current);
        }
    }

    if (factor < -1) {
        if (searchKey > current->right->key) {
            return rotateLeft(current);
        } else if (searchKey < current->right->key) {
            current->right = rotateRight(current->right);
            return rotateLeft(current);
        }
    }

    return current;
}

void AVLIndex::Add(const int i, const string &Key) {
    root = insert(root, i, Key);
}

```

BST.cpp 41%

```
#include "BST.hpp"

void BSTIndex::CreateIndex(vector<FileSystemEntry> &Data, const string &IndexType)
{
    int total = (int)Data.size();
    for (int idx = 0; idx < total; idx++) {
        if (IndexType == "Filename") {
            Add(idx, Data[idx].Filename);
        } else if (IndexType == "modifiedOn") {
            Add(idx, Data[idx].modifiedOn);
        }
    }
}

void BSTIndex::Add(const int i, const string &Key) {
    root = insert(root, i, Key);
}

TreeNode* BSTIndex::insert(TreeNode* curr, int fileIdx, const string& val) {
    if (curr == nullptr) {
        return new TreeNode(val, fileIdx);
    }

    if (val < curr->key) {
        curr->left = insert(curr->left, fileIdx, val);
    } else if (val > curr->key) {
        curr->right = insert(curr->right, fileIdx, val);
    } else {
        curr->indices.push_back(fileIdx);
    }
    return curr;
}
```

```
}

std::pair<std::vector<int>, int> BSTIndex::Search(const std::string &Key) {
    TreeNode* curr = root;
    int nodesVisited = 0;
    while (curr) {
        nodesVisited++;
        if (Key == curr->key) {
            return {curr->fileIndices, nodesVisited};
        } else if (Key < curr->key) {
            curr = curr->left;
        } else {
            curr = curr->right;
        }
    }
    return {{}, nodesVisited};
}

std::pair<std::vector<int>, int> BSTIndex::SearchRange(const std::string &key1, const std::string &key2) {
    std::vector<int> results;
    int nodesVisited = 0;
    rangeSearch(root, key1, key2, results, nodesVisited);
    return {results, nodesVisited};
}

void BSTIndex::rangeSearch(TreeNode* node, const std::string& key1, const std::string& key2, std::vector<int>& results, int& nodesVisited) {
    if (!node) return;
    nodesVisited++;
    if (key1 < node->key) {
        rangeSearch(node->left, key1, key2, results, nodesVisited);
    }
    if (node->key >= key1 && node->key <= key2) {
        results.insert(results.end(), node->fileIndices.begin(), node->fileIndices.end());
    }
    if (key2 > node->key) {
        rangeSearch(node->right, key1, key2, results, nodesVisited);
    }
}

```

FileExplorer.cpp 38%

```
#include "FileExplorer.hpp"
#include <fstream>
#include <set>

FileExplorer::FileExplorer(const string &type, const string &csv_file) {
    fs = new FileSystem();
    fs->LoadFilesFromCsv(csv_file);

    if (type == "BST") {
        NameIndex = new BSTIndex();
        DateIndex = new BSTIndex();
```

```

pair<vector<int>, int> BSTIndex::Search(const string &Key) {
    int totalVisited = 0;
    TreeNode* found = search(root, Key, totalVisited);
    if (found != nullptr) {
        return {found->indices, totalVisited};
    }
    vector<int> empty;
    return {empty, totalVisited};
}

TreeNode* BSTIndex::search(TreeNode* n, const string& k, int& count) const {
    if (n == nullptr) return nullptr;

    count = count + 1;
    if (k == n->key) {
        return n;
    }

    if (k < n->key) {
        return search(n->left, k, count);
    } else {
        return search(n->right, k, count);
    }
}

void BSTIndex::clear(TreeNode* n) {
    if (n == nullptr) return;
    clear(n->left);
    clear(n->right);
    delete n;
}

pair<vector<int>, int> BSTIndex::RangeSearch(const string& a, const string& b) {
    vector<int> list;
    int steps = 0;
    rangeSearch(root, a, b, list, steps);
    return {list, steps};
}

void BSTIndex::rangeSearch(TreeNode* n, const string& low, const string& high, vector<int>& out, int& count) const {
    if (n == nullptr) return;

    count++;

    if (n->key > low) {
        rangeSearch(n->left, low, high, out, count);
    }

    if (n->key >= low && n->key <= high) {
        int sz = (int)(n->indices.size());
        for (int i = 0; i < sz; i++) {
            out.push_back(n->indices[i]);
```

```
    } else {
        NameIndex = new AVLIndex();
        DateIndex = new AVLIndex();
    }

    NameIndex->CreateIndex(fs->entries, "Filename");
    DateIndex->CreateIndex(fs->entries, "modifiedOn");
}

FileExplorer::~FileExplorer() {
    delete NameIndex;
    delete DateIndex;
    delete fs;
}

void FileExplorer::FindByName(const string &filename, const string &output_path) {
    auto result = NameIndex->Search(filename);
    ofstream outfile(output_path);
    outfile << "Number of nodes traversed: " << result.second << "\n";
    for (int idx : result.first) {
        outfile << fs->entries[idx].path << "\n";
    }
}

void FileExplorer::FindByDate(const string &date, const string &output_path) {
    auto result = DateIndex->Search(date);
    ofstream outfile(output_path);
    outfile << "Number of nodes traversed: " << result.second << "\n";
    for (int idx : result.first) {
        outfile << fs->entries[idx].path << "\n";
    }
}

void FileExplorer::FindByNameAndDate(const string &filename, const string &date,
const string &output_path) {
    auto nameResult = NameIndex->Search(filename);
    auto dateResult = DateIndex->Search(date);

    set<int> nameSet(nameResult.first.begin(), nameResult.first.end());
    vector<int> intersection;
    for (int idx : dateResult.first) {
        if (nameSet.count(idx)) {
            intersection.push_back(idx);
        }
    }

    ofstream outfile(output_path);
    outfile << "Number of nodes traversed: " << nameResult.second +
dateResult.second << "\n";
    for (int idx : intersection) {
        outfile << fs->entries[idx].path << "\n";
    }
}

    }

    if (n->key < high) {
        rangeSearch(n->right, low, high, out, count);
    }
}
```

FileExplorer.cpp 31%

```
#include "FileExplorer.hpp"
#include <fstream>

FileExplorer::FileExplorer(const string &t, const string &f) {
    fs = new FileSystem();
    fs->LoadFilesFromCsv(f);

    if (t == "BST") {
        NameIndex = new BSTIndex();
        DateIndex = new BSTIndex();
    } else {
        NameIndex = new AVLIndex();
        DateIndex = new AVLIndex();
    }

    NameIndex->CreateIndex(fs->entries, "Filename");
    DateIndex->CreateIndex(fs->entries, "modifiedOn");
}

FileExplorer::~FileExplorer() {
    delete NameIndex;
    delete DateIndex;
    delete fs;
}

void dumpToFilesystem(const vector<int>& list, int numVisited, const
vector<FileSystemEntry>& info, const string& pathOut) {
    ofstream writer(pathOut);
    writer << "Number of nodes traversed: " << numVisited << "\n";

    int sz = (int)list.size();
    for (int i = 0; i < sz; i++) {
        int pos = list[i];
        string entryPath = info[pos].path;
        if (entryPath.length() > 0 && entryPath[0] == '/') {
            entryPath = entryPath.substr(1);
        }
        writer << entryPath << "\n";
    }
}

void FileExplorer::FindByName(const string &name, const string &out) {
    pair<vector<int>, int> results = NameIndex->Search(name);
    dumpToFilesystem(results.first, results.second, fs->entries, out);
```

```
void FileExplorer::FindByNameAndSize(const string &filename, const int size, const string &output_path) {
    auto result = NameIndex->Search(filename);
    vector<int> filtered;
    for (int idx : result.first) {
        if (fs->entries[idx].Size == size) {
            filtered.push_back(idx);
        }
    }

    ofstream outfile(output_path);
    outfile << "Number of nodes traversed: " << result.second << "\n";
    for (int idx : filtered) {
        outfile << fs->entries[idx].path << "\n";
    }
}

void FileExplorer::FindFilesCreatedDuring(const string &date1, const string &date2, const string &output_path) {
    auto result = DateIndex->SearchRange(date1, date2);
    ofstream outfile(output_path);
    outfile << "Number of nodes traversed: " << result.second << "\n";
    for (int idx : result.first) {
        outfile << fs->entries[idx].path << "\n";
    }
}

```

FileSystem.cpp 0%

```
#include "FileSystem.hpp"

#include <ctime>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <map>
#include <sstream>
#include <vector>

using namespace std;

void FileSystem::LoadFilesFromCsv(const string &csv_file) {
    ifstream file(csv_file);
    if (!file.is_open()) {
        cerr << "Error opening file " << csv_file << endl;
    }
    string line;

    getline(file, line);  // Skip header
    while (getline(file, line)) {
        stringstream ss(line);
        FileSystemEntry entry;
        getline(ss, entry.Filename, ',');
        getline(ss, entry.Type, ',');
```

```
}

void FileExplorer::FindByDate(const string &day, const string &out) {
    pair<vector<int>, int> results = DateIndex->Search(day);
    dumpToFilesystem(results.first, results.second, fs->entries, out);
}

void FileExplorer::FindByNameAndDate(const string &name, const string &day, const string &out) {
    pair<vector<int>, int> r1 = NameIndex->Search(name);
    pair<vector<int>, int> r2 = DateIndex->Search(day);

    vector<int> both;
    for (int a = 0; a < (int)r1.first.size(); a++) {
        for (int b = 0; b < (int)r2.first.size(); b++) {
            if (r1.first[a] == r2.first[b]) {
                both.push_back(r1.first[a]);
                break;
            }
        }
    }

    dumpToFilesystem(both, r1.second + r2.second, fs->entries, out);
}

void FileExplorer::FindByNameAndSize(const string &name, const int bytes, const string &out) {
    pair<vector<int>, int> results = NameIndex->Search(name);
    vector<int> refined;
    for (int i = 0; i < (int)results.first.size(); i++) {
        int idx = results.first[i];
        if (fs->entries[idx].Size == bytes) {
            refined.push_back(idx);
        }
    }
    dumpToFilesystem(refined, results.second, fs->entries, out);
}

void FileExplorer::FindFilesCreatedDuring(const string &dStart, const string &dEnd, const string &out) {
    pair<vector<int>, int> output = DateIndex->RangeSearch(dStart, dEnd);
    dumpToFilesystem(output.first, output.second, fs->entries, out);
}
```

FileSystem.cpp 0%

```
#include "FileSystem.hpp"

#include <string>
#include <vector>
#include <fstream>
#include <iostream>

using namespace std;
```

```
        ss >> entry.Size;
        ss.ignore();
        // Read and parse timestamps
        string accessedOnStr, modifiedOnStr;
        getline(ss, accessedOnStr, ',');
        getline(ss, modifiedOnStr, ',');
        entry.modifiedOn = ParseTimestamp(modifiedOnStr);
        entry.accessedOn = ParseTimestamp(accessedOnStr);

        ss.ignore();
        getline(ss, entry.path, ',');
        entries.push_back(entry);
    }
    file.close();
}

void FileSystem::PrintFileEntry() {
    // print the file entry
    for (const auto &entry : entries) {
        // print each column on a new line
        cout << "Filename: " << entry.Filename << "\n";
        cout << "Type: " << entry.Type << "\n";
        cout << "Size:" << entry.Size << "\n";
        cout << "accessedOn: " << entry.accessedOn << "\n";
        cout << "modifiedOn: " << entry.modifiedOn << "\n";
        cout << "path: " << entry.path << "\n";
        cout << "\n";
    }
}


string FileSystem::ParseTimestamp(const string &timestampStr) {
    // Convert timestamp string to tm struct
    struct tm tm = {};
    stringstream ss(timestampStr);
    ss >> get_time(&tm, "%Y-%m-%dT%H:%M:%S");
    if (ss.fail()) {
        cerr << "Error parsing timestamp: " << timestampStr << endl;
        return "";  // Return empty string if parsing fails
    }

    // Format tm struct to desired date string
    stringstream formatted_date;
    formatted_date << std::put_time(&tm, "%Y-%m-%d");
    return formatted_date.str();
}
```

```
int convertToNum(string str) {
    int val = 0;
    int len = (int)str.length();
    for (int idx = 0; idx < len; idx++) {
        char ch = str[idx];
        if (ch >= '0' && ch <= '9') {
            val = val * 10 + (ch - '0');
        }
    }
    return val;
}

void FileSystem::LoadFilesFromCsv(const string &path) {
    ifstream input(path);
    string row;

    if (getline(input, row)) {
        // Skip header
    }

    while (getline(input, row)) {
        if (row.length() == 0) continue;

        FileSystemEntry item;
        string cols[6];
        int colIdx = 0;
        string buffer = "";

        for (int k = 0; k < (int)row.length(); k++) {
            char currentCh = row[k];
            if (currentCh == ',' && colIdx < 5) {
                cols[colIdx++] = buffer;
                buffer = "";
            } else {
                buffer += currentCh;
            }
        }
        cols[colIdx] = buffer;

        if (colIdx >= 5) {
            item.Filename = cols[0];
            item.Type = cols[1];
            item.Size = convertToNum(cols[2]);
            item.accessedOn = ParseTimestamp(cols[3]);
            item.modifiedOn = ParseTimestamp(cols[4]);
            item.path = cols[5];
            entries.push_back(item);
        }
    }
    input.close();
}

void FileSystem::PrintFileEntry() {
    int count = (int)entries.size();
    for (int i = 0; i < count; i++) {
```

```
    for (int j = 0; j < count; j++) {
        FileSystemEntry& f = entries[j];
        cout << "Filename: " << f.Filename << "\n";
        cout << "Type: " << f.Type << "\n";
        cout << "Size:" << f.Size << "\n";
        cout << "accessedOn: " << f.accessedOn << "\n";
        cout << "modifiedOn: " << f.modifiedOn << "\n";
        cout << "path: " << f.path << "\n";
        cout << "\n";
    }
}

string FileSystem::ParseTimestamp(const string &s) {
    if (s.length() < 10) return "";
    string date = "";
    for (int i = 0; i < 10; i++) {
        date += s[i];
    }
    return date;
}
```